Computer Vision

Assignment 2: Image Processing

Pablo Mesejo and Siham Tabik
Universidad de Granada
Departamento de Ciencias de la Computación e Inteligencia Artificial



UNIVERSIDAD DE GRANADA



Index

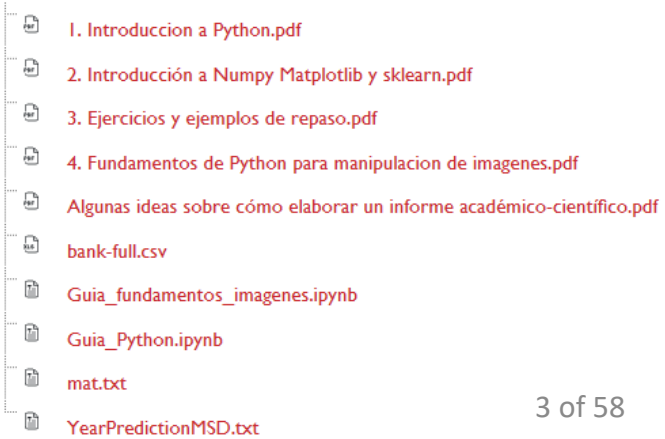
- Submission rules
- Brief review of convolutions and filters
- Description and analysis of Assignment 2

Submission Rules

- Only one .ipynb file is submitted, directly **integrating code, results, analysis and discussion**.

- The template/statement is available in PRADO, as well as an example of a correctly solved exercise: Ejemplo_Otsu.ipynb.

- You also have supplementary materials!
  https://pradogrado2627.ugr.es/mod/folder/view.php?id=83733



Submission Rules

- Read very carefully the rules in the template:
  - Upload only the .ipynb file to PRADO. Do not upload images to PRADO!
  - Reading images or any input file: "`/content/drive/MyDrive/images`"
  - Do not write anything into disk (i.e, do not write anything in Google Drive).

Submission

- Deadline: 15 November
  - Late submissions will not be accepted (there are no flexible deadlines).
  - However, the deadline itself is negotiable (it may be adjusted in advance if necessary and agreed upon beforehand).
- Maximum score: 10 points
- Submission site: PRADO

- **Explanation/discussion/analysis accompanying code and results is essential.**

Goals

- To learn how to implement and apply **convolution filters** and, in particular, smoothing and derivative image filters.
- To show how, using linear filtering techniques, it is possible to extract relevant information from an image to allow its **interpretation**.

- This is an assignment very oriented towards **image processing**.

Exercise 1

- Key ideas:
  - Initially, you can only use basic OpenCV functions
    - You must know how certain operations are performed at a low level.
    - Unless we explicitly tell you otherwise, you cannot use, for example, `GaussianBlur()` or `pyrDown()`. You have to do it by hand.

Exercise 1

- Key ideas:
  - “implement efficiently”
    - Use only 1D convolutions.
      - We want you to make use of “separability”, according to which a 2D convolution can be reduced to two 1D convolutions.
      - OpenCV function `sepFilter2D()`

Exercise 1.A

- **Perform 2D convolutions using the 1D masks created in the previous assignment.**

- You must use *cv2.sepFilter2D()* to apply the separable filters on the image.

Exercise 1.A

- The idea is to create a function that, **given an image**, **a sigma**, **and** a list with two elements (representing **the order of derivative applied by rows and columns**, respectively) **provides a new image as result of the convolution operation**.

```
my2DConv(im, sigma, orders)
```

Exercise 1.A

◆ sepFilter2D()

```
void cv::sepFilter2D ( InputArray  src,
                       OutputArray dst,
                       int         ddepth,
                       InputArray  kernelX,
                       InputArray  kernelY,
                       Point       anchor = Point(-1,-1) ,
                       double      delta = 0 ,
                       int         borderType = BORDER_DEFAULT
                     )
```

Python:

cv.sepFilter2D(src, ddepth, kernelX, kernelY[, dst[, anchor[, delta[, borderType]]]]) -> dst

`#include <opencv2/imgproc.hpp>`

Applies a separable linear filter to an image.

The function applies a separable linear filter to the image. That is, first, every row of src is filtered with the 1D kernel kernelX. Then, every column of the result is filtered with the 1D kernel kernelY. The final result shifted by delta is stored in dst .

Parameters

src	Source image.
dst	Destination image of the same size and the same number of channels as src .
ddepth	Destination image depth, see **combinations**
kernelX	Coefficients for filtering each row.
kernelY	Coefficients for filtering each column.
anchor	Anchor position within the kernel. The default value $(-1,-1)$ means that the anchor is at the kernel center.
delta	Value added to the filtered results before storing them.
borderType	Pixel extrapolation method, see **BorderTypes**. **BORDER_WRAP** is not supported.

See also

filter2D, **Sobel**, **GaussianBlur**, **boxFilter**, **blur**

https://docs.opencv.org/4.6.0/d4/d86/group__imgproc__filter.html#ga910e29ff7d7b105057d1625a4bf6318d

```
if orders==[0,0]:
    return cv2.sepFilter2D(im,-1,maskG,maskG)
elif
    ...
else:
    print('error in order of derivative')
```

Note

when ddepth=-1, the output image will have the same depth as the source.

This refers to the type used. **If you set -1 it will use the type of the input image. If uint8, it will saturate to zero and only positive values will appear**. To solve this, you can use cv2.CV_64F

Exercise 1.A

The **1st derivative of a 2D Gaussian** is separable:

$$
\begin{aligned}
G(x,y) &= \frac{1}{2\pi\sigma^2}\exp\left(\frac{-(x^2+y^2)}{2\sigma^2}\right) \\
&= \frac{1}{\sqrt{2\pi\sigma^2}}\exp\left(-\frac{x^2}{2\sigma^2}\right)\cdot\frac{1}{\sqrt{2\pi\sigma^2}}\exp\left(-\frac{y^2}{2\sigma^2}\right) \\
&= G_h(x)\cdot G_v(y) \\
\frac{\partial G(x,y)}{\partial x} &= -\frac{x}{\sigma^2}\cdot\frac{1}{2\pi\sigma^2}\exp\left(-\frac{x^2}{2\sigma^2}\right)\exp\left(-\frac{y^2}{2\sigma^2}\right) \\
&= G'_h(x)\cdot G_v(y)
\end{aligned}
$$

Derivative of horizontal 1D Gaussian kernel (row-wise)

1D vertical Gaussian kernel (column-wise)

Smooth in one direction and calculate differences in the other

This allows us to know **which 1D kernels to apply, and in which order, to calculate the derivatives of an image**. In the example above, the first derivative in X.

Exercise 1.B

And the **2nd derivatives of Gaussian** as well:

$$\frac{\partial^2 G}{\partial x^2} + \frac{\partial^2 G}{\partial y^2} = \boxed{G_h''(x) \cdot G_v(y)} + \boxed{G_h(x) \cdot G_v''(y)}$$

2nd derivative in X

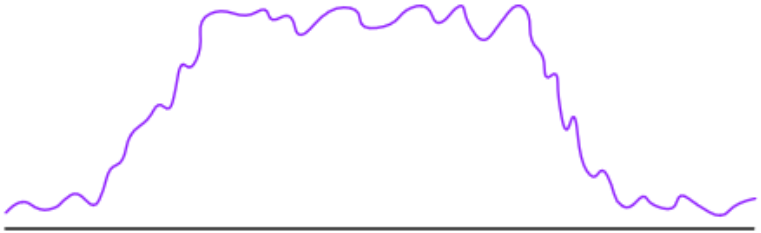
2nd derivative in Y

convolution in one direction with the 2nd derivative of Gaussian and then, in the other direction, convolution with Gaussian.

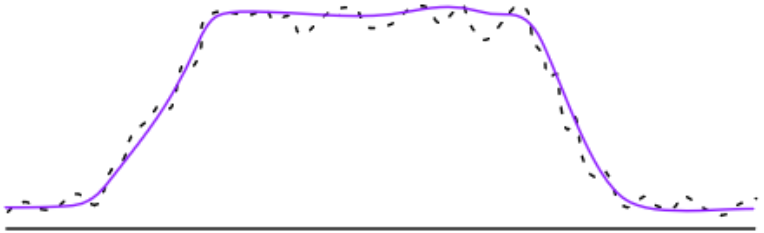
convolution in one direction with Gaussian and then, in the other direction, convolution with the 2nd derivative of Gaussian.

Note: what you see in this slide ($\frac{\partial^2 G}{\partial x^2} + \frac{\partial^2 G}{\partial y^2}$) is, in fact, the **Laplacian of Gaussian**

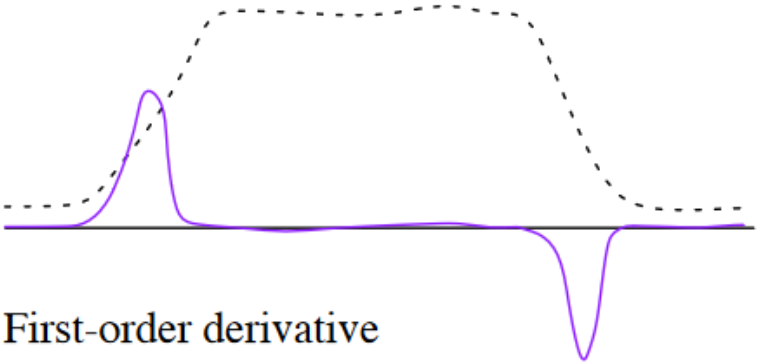
Edge detection using first- and second-order derivatives



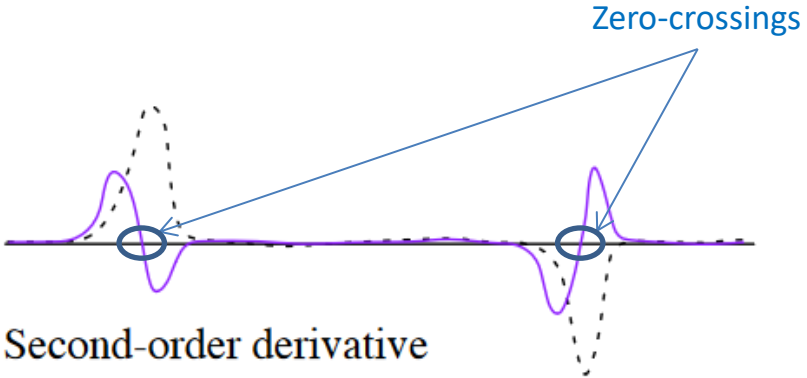
Intensity profile of an input image



After noise removal



First-order derivative



Second-order derivative

Main advantage of second derivatives?

They provide a **convenient way to localize edges through zero-crossings**. But be careful with noise!

Zero-crossings provide a **simple criterion for edge localization** (see https://www.cs.toronto.edu/~mangas/teaching/320/slides/CSC320L05.pdf)

https://www.cs.auckland.ac.nz/~rklette/TeachAuckland.html/775/pdfs/D06-LocalOp1.pdf

Derivatives and Edges

We'll compute these in Exercise 1.B

Horizontal derivative

$$\mathbf{G_x} = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}$$

Vertical derivative

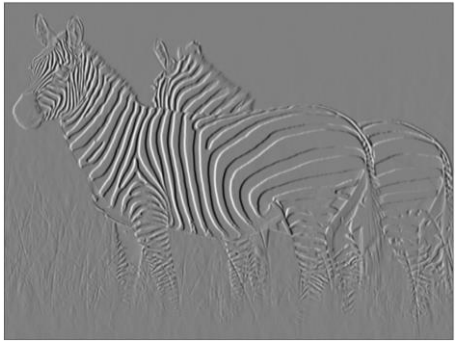
$$\mathbf{G_y} = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$$

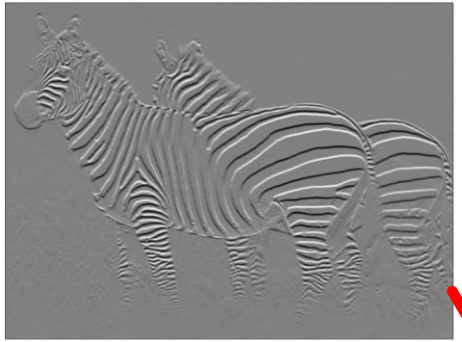
Gradient magnitude

$$\mathbf{G} = \sqrt{\mathbf{G_x}^2 + \mathbf{G_y}^2}$$

Gradient orientation

$$\mathbf{\Theta} = \operatorname{atan2}(\mathbf{G}_y, \mathbf{G}_x)$$





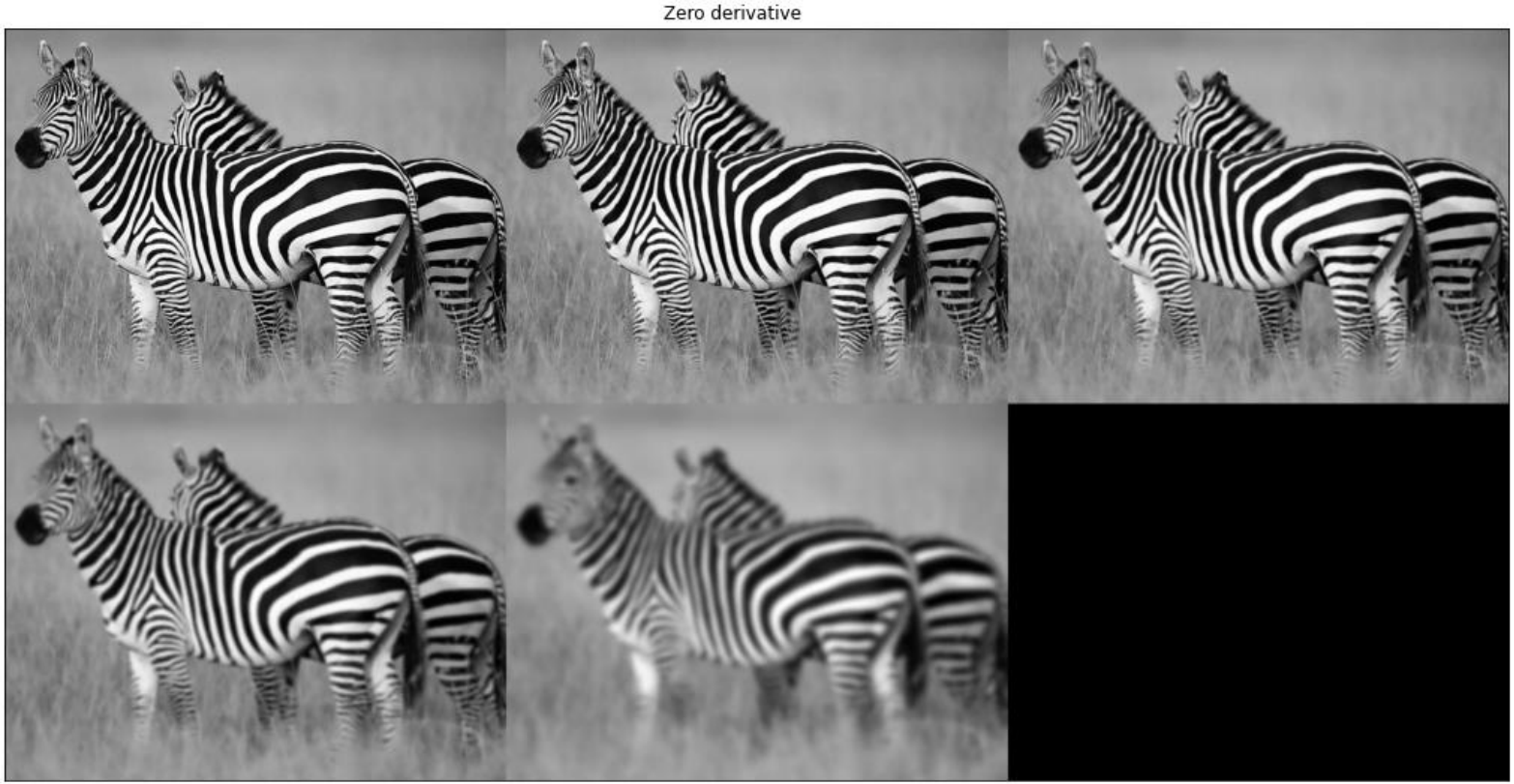




Exercise 1.A

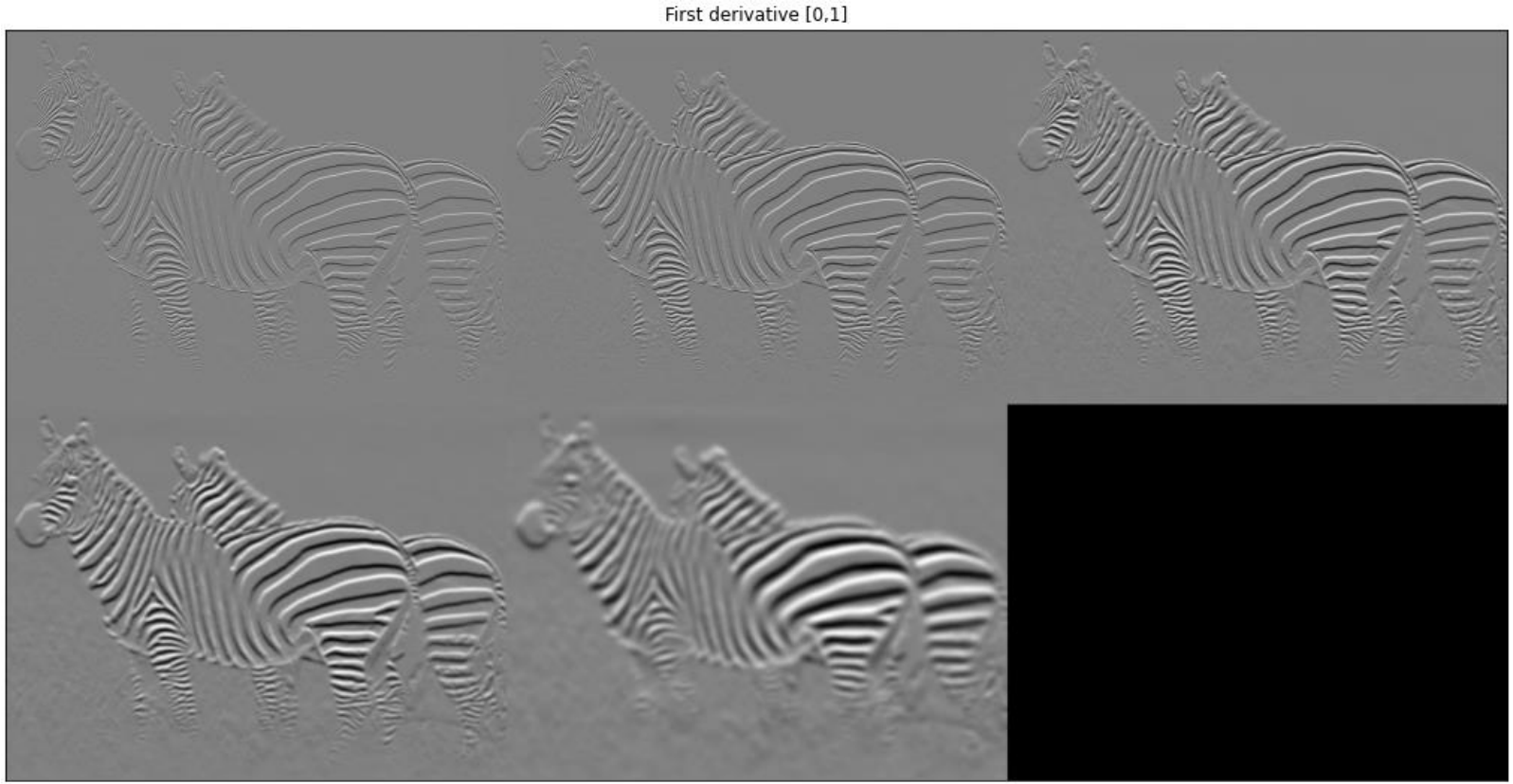
- Important notes:
  - Everything is done in **greyscale**: *cv.imread(filename,0)*
    - Working in RGB colour is just doing in triplicate what is done in a single band.
  - Operate in **float** to avoid precision problems and saturation of negative values!

Exercise 1.A



Exercise 1.A

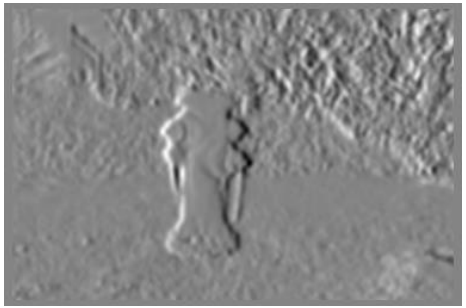
Are these derivatives on X or Y? 🤔



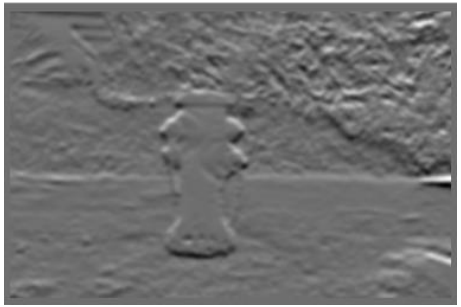
Exercise 1.B

- **Calculate the gradient and Laplacian of a given image. We start with the gradient:**

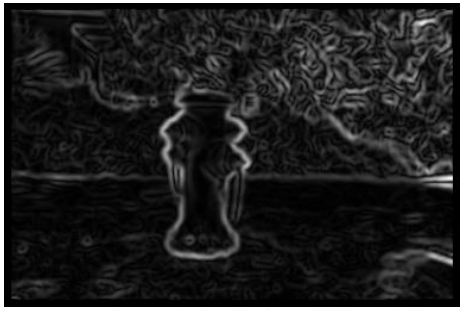




$$g_x = I * G_y * G'_x$$



$$g_y = I * G_x * G'_y$$



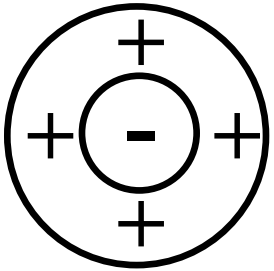
$$|\nabla I| = \sqrt{g_x^2 + g_y^2}$$

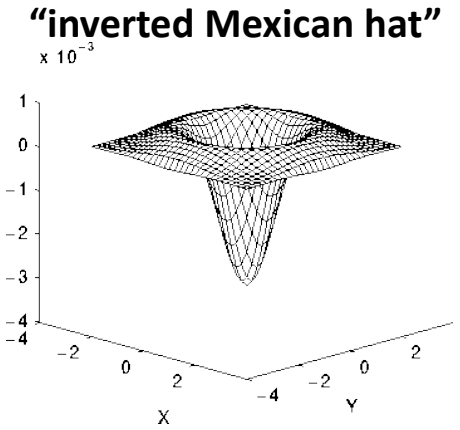
Extracted from "Image Filtering and Edge Detection" de Stan Birchfield, Clemson University

Exercise 1.B

- Example of a 2D 3x3 Laplacian kernel:

```
 0  1  0
 1 -4  1
 0  1  0
```





- Is it separable?
  - First, what is necessary for a kernel to be separable?

Exercise 1.B

- What is necessary for a kernel to be separable?

  A 2D kernel is *separable* if and only if all non-zero rows/columns are scalar multiples of one another.

- This filter $\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$, therefore, is NOT:

  $\alpha \cdot [0, 1, 0] + \beta \cdot [1, -4, 1] = [0, 0, 0] \leftrightarrow \alpha = 0 \wedge \beta = 0$

  → They are linearly independent vectors!!

Exercise 1.B

```
G = np.array([[0,1,0],[1,-4,1],[0,1,0]])
array([[ 0,  1,  0],
       [ 1, -4,  1],
       [ 0,  1,  0]])


np.linalg.matrix_rank(G)
2

G = np.outer([1,2,1],[1,2,1])
np.linalg.matrix_rank(G)
1

G = np.outer([1,2,1],[-1,0,1])
np.linalg.matrix_rank(G)
1

np.linalg.matrix_rank(np.outer(gaussianMask1D(1, None, 0), gaussianMask1D(1, None, 2)))
1
```

If the rank is not 1 → it is not directly separable!

Remember that **the rank of a matrix is the maximum number of linearly independent columns/rows**.

[[1 2 1]
[2 4 2]
[1 2 1]]
if α = 2 and β = -1 → α·[1, 2, 1] + β·[2, 4, 2] = [0, 0, 0]

[[-1 0 1]
[-2 0 2]
[-1 0 1]]
if α = -2 and β = 1 → α·[-1, 0, 1] + β·[-2, 0, 2] = [0, 0, 0]

In the case of the LoG, we have two matrices with rank=1 → **We have two separable 2D filters!!!!**

Exercise 1.B

- We can calculate LoG as the sum of 2D convolutions that are separable
  - Because the Gaussian and its derivatives are!!!

    → we need 4 1D convolutions (and that is still less computationally expensive than doing a single 2D convolution; unless the kernel is very small)

if the kernel is 7×7 we need 49 multiplications per pixel for the 2D kernel, or 4·7=28 multiplications per pixel for the four 1D kernels; this difference grows as the kernel gets larger

Interesting references on the LoG: https://stackoverflow.com/questions/53544983/how-is-laplacian-filter-calculated/53545480#53545480 and https://www.crisluengo.net/archives/1099/

Exercise 1.B

The Laplacian operator is the divergence of the gradient, and is given by the sum of the second order derivatives (i.e. the trace of the Hessian matrix):

$$\nabla^2 I = \nabla \cdot \nabla I = \begin{bmatrix} \frac{\partial}{\partial x} & \frac{\partial}{\partial y} \end{bmatrix} \begin{bmatrix} \frac{\partial I}{\partial x} & \frac{\partial I}{\partial y} \end{bmatrix}^T = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}$$

$$\frac{\partial^2 (I * G)}{\partial x^2} + \frac{\partial^2 (I * G)}{\partial y^2}$$

Laplacian of an image smoothed with a Gaussian kernel

$$\nabla^2 (I * G)$$

Differentiation property of convolution

$$\frac{\partial^2 (f * g)}{\partial x^2} = f * \frac{\partial^2 g}{\partial x^2}$$

$$\frac{\partial^2 (f * g)}{\partial y^2} = f * \frac{\partial^2 g}{\partial y^2}$$

$$I * \frac{\partial^2 G}{\partial x^2} + I * \frac{\partial^2 G}{\partial y^2}$$

Sum of convolutions with the second derivatives of the Gaussian

$$I * \frac{\partial^2 G}{\partial x^2} + I * \frac{\partial^2 G}{\partial y^2}$$

Distributive property of convolution

$$f * (g + h) = f * g + f * h$$

$$I * \left(\frac{\partial^2 G}{\partial x^2} + \frac{\partial^2 G}{\partial y^2} \right) = I * \nabla^2 G$$

Image convolved with the Laplacian of a Gaussian

$$I * \nabla^2 G$$

Exercise 1.B

How do we calculate $I * \frac{\partial^2 G}{\partial x^2} + I * \frac{\partial^2 G}{\partial y^2}$?

We know that the **1st derivative of a 2D Gaussian** is separable:

$$
\begin{aligned}
G(x,y) &= \frac{1}{2\pi\sigma^2}\exp\left(\frac{-(x^2+y^2)}{2\sigma^2}\right) \\
&= \frac{1}{\sqrt{2\pi\sigma^2}}\exp\left(-\frac{x^2}{2\sigma^2}\right)\cdot\frac{1}{\sqrt{2\pi\sigma^2}}\exp\left(-\frac{y^2}{2\sigma^2}\right) \\
&= G_h(x)\cdot G_v(y) \\
\frac{\partial G(x,y)}{\partial x} &= -\frac{x}{\sigma^2}\cdot\frac{1}{2\pi\sigma^2}\exp\left(-\frac{x^2}{2\sigma^2}\right)\exp\left(-\frac{y^2}{2\sigma^2}\right) \\
&= G'_h(x)\cdot G_v(y)
\end{aligned}
$$

1D horizontal Gaussian kernel derivative

1D vertical Gaussian kernel

Smooth in one direction, differentiate in the other

Exercise 1.B

$$\frac{\partial^2 G}{\partial x^2} + \frac{\partial^2 G}{\partial y^2} = G''_h(x) \cdot G_v(y) + G_h(x) \cdot G''_v(y)$$

convolution in one direction with the 2nd derivative of Gaussian and, then, in the other, convolution with Gaussian.

convolution in one direction with Gaussian and, then, in the other, convolution with the 2nd derivative of Gaussian.

Exercise 1.B

- **Laplacian of Gaussian**

$$L = \sigma^2 \left(G_{xx}(x, y, \sigma) + G_{yy}(x, y, \sigma) \right)$$

(Laplacian)

$$\frac{\partial^2 G}{\partial x^2} + \frac{\partial^2 G}{\partial y^2} = G''_h(x) \cdot G_v(y) + G_h(x) \cdot G''_v(y)$$

1) dxx: the convolution by rows with the second derivative of the Gaussian and, then, by columns, the convolution with the Gaussian.

2) dyy: the convolution by rows with the Gaussian and, then, by columns, the convolution with the second derivative of the Gaussian.

3) We add dxx and dyy

4) We multiply the result of the sum by σ^2

Actually, you only have to scale/normalize the masks obtained in Assignment 1, **multiplying them by σ (1st derivative) or σ² (2nd derivative)**

Useful reference: https://www.crisluengo.net/archives/1099/

Exercise 1.C

- You have to implement your own separable convolution function (`def convolve1D(image,kernel)`) and check the differences/similarities of the results obtained with `cv2.Sobel()`

Exercise 1.C

```
CONVOLVE2D(f, g)
  Input:  2D image f_{width × height}, 2D kernel g_{w × h}
  Output: the 2D convolution of f and g
1  h̃ ← ⌊(h − 1)/2⌋
2  w̃ ← ⌊(w − 1)/2⌋
3  for y ← 0 to height − 1 do
4        for x ← 0 to width − 1 do
5              val ← 0
6              for j ← 0 to h − 1 do
7                    for i ← 0 to w − 1 do
8                          val ← val + g(i, j) * f(x + w̃ − i, y + h̃ − j)
9              h(x, y) ← val
10 return h
```

- Convolution cannot be done “in place”
- **Output and input image must be completely separated**
  - The new values calculated by sliding the window/kernel cannot be used in subsequent calculations.
- Otherwise the calculations are wrong

Exercise 1.C

ConvolveSeparable(I, g_h, g_v)

Input: 2D image $I_{\{width \times height\}}$, 1D kernels g_h and g_v of length w
Output: the 2D convolution of I and $g_v \circledast g_h$

```
1  ▷ convolve horizontal
2  for y ← 0 to height − 1 do
3        for x ← w̃ to width − 1 − w̃ do
4              val ← 0
5              for i ← 0 to w − 1 do
6                    val ← val + g_h[i] ∗ I(x + w̃ − i, y)
7              tmp(x, y) ← val
8  ▷ convolve vertical
9  for y ← w̃ to height − 1 − w̃ do
10       for x ← 0 to width − 1 do
11             val ← 0
12             for i ← 0 to w − 1 do
13                   val ← val + g_v[i] ∗ tmp(x, y + w̃ − i)
14             out(x, y) ← val
15 return out
```

Extracted from "Image Filtering and Edge Detection" by Stan Birchfield, Clemson University

Interesting reference:
http://www.songho.ca/dsp/convolution/convolution2d_separable.html
It helps to better understand 2D convolution from 1D kernels.

In essence, it consists of **going through the whole image, pixel by pixel, first doing the convolution by rows and then, on the result, applying the convolution of the 1D kernel by columns**.

Exercise 1.C

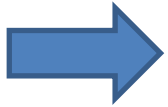
We combine the information from different rows
(kernel Y - Coefficients for filtering each column)

We combine the information from different columns
(kernel X - Coefficients for filtering each row)

```
[1]
[2]
[1]
```

```
[ 0,  0,  0,  0,  0]
[ 0, 10, 20, 30,  0]
[ 0, 40, 50, 60,  0]
[ 0, 70, 80, 90,  0]
[ 0,  0,  0,  0,  0]
```

```
[1, 2, 1]
```



```
[  0,   0,   0,   0,   0]
[  0,  60,  90, 120,   0]
[  0, 160, 200, 240,   0]
[  0, 180, 210, 240,   0]
[  0,   0,   0,   0,   0]
```



Input image

```
[ 0,  0,  0,  0,  0]
[ 0, 10, 20, 30,  0]
[ 0, 40, 50, 60,  0]
[ 0, 70, 80, 90,  0]
[ 0,  0,  0,  0,  0]
```

Kernel

```
[1, 2, 1]
```

$$\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$$

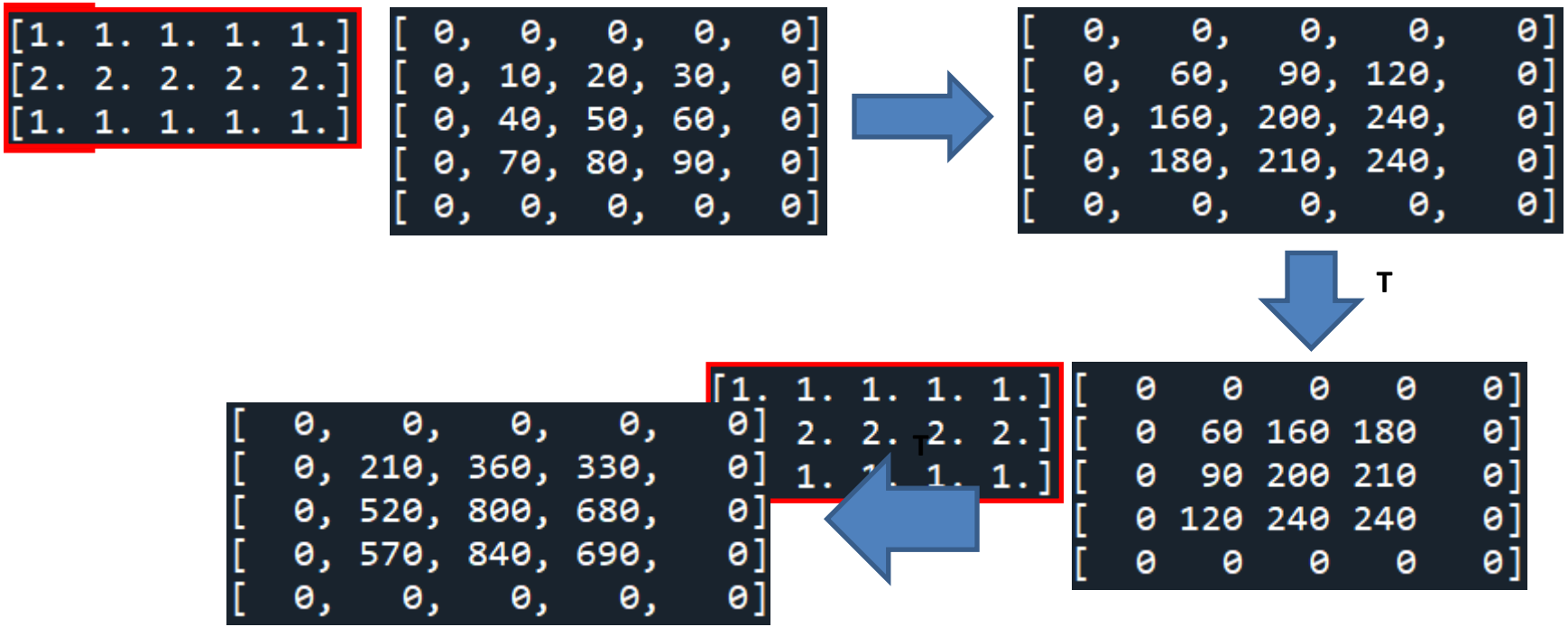
Separable Kernel

http://www.songho.ca/dsp/convolution/convolution2d_separable.html

```
[  0,   0,   0,   0,   0]
[  0, 210, 360, 330,   0]
[  0, 520, 800, 680,   0]
[  0, 570, 840, 690,   0]
[  0,   0,   0,   0,   0]
```

Exercise 1.C

It is not necessary to always go through all the rows and columns (4 for-loops)!!!
For each 1D convolution, we only need to traverse the rows once!!!



Exercise 1

Masks discretization (Assignment 1)
+
Separable convolutions

Gaussian smoothing

Edge detection/enhancement by means of Gaussian derivatives, and Laplacian of Gaussian

Exercise 2

- **2.A Generate a 4-level Gaussian pyramid representation**
- **2.B Generate a 4-level Laplacian pyramid representation**
- **2.C Use Laplacian pyramid to recover the original image**
- **2.D Some questions about pyramids**

Exercise 2

- Importance of **image pyramids**:
  - We are used to work with fixed size images.
  - Many times, we need **to work with an image at different resolutions**.
    - For example, if we are looking for something specific, such as a face, and we do not know *a priori* the size of the object we are looking for.
    - Or if we want to subsample an image avoiding aliasing.
    - Or if we need to compress an image.

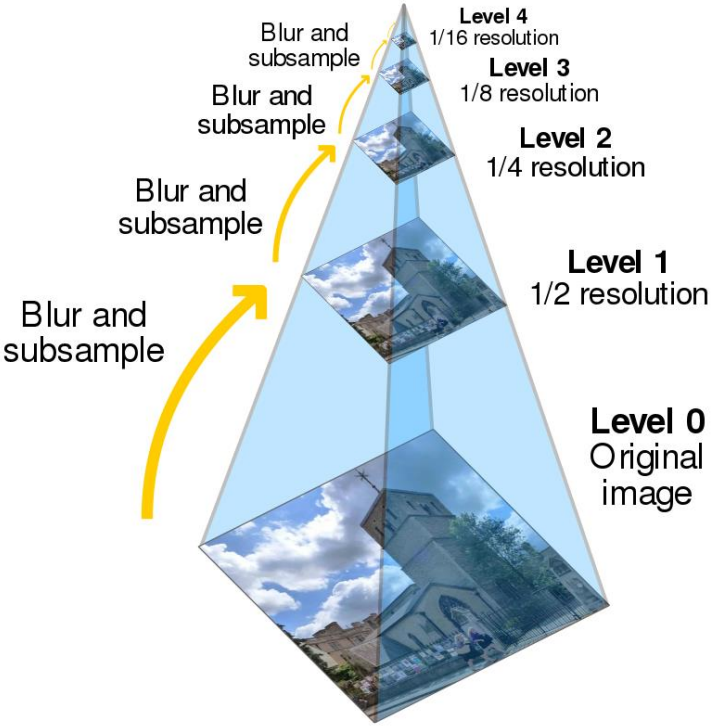
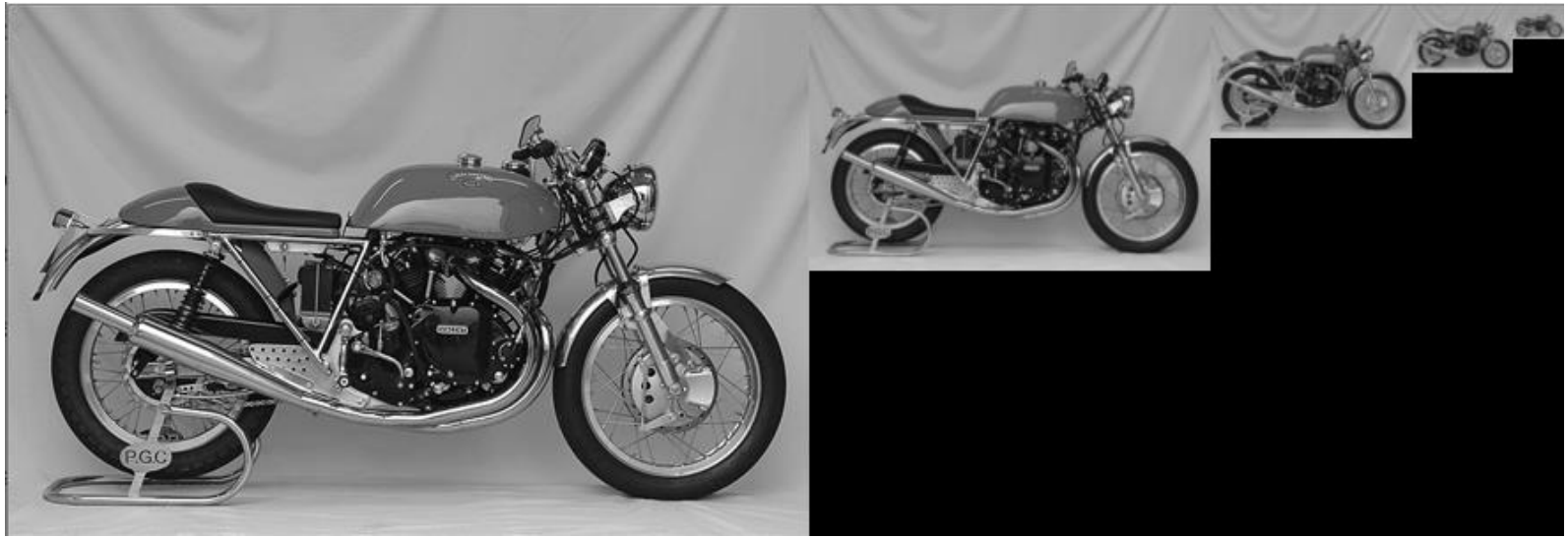


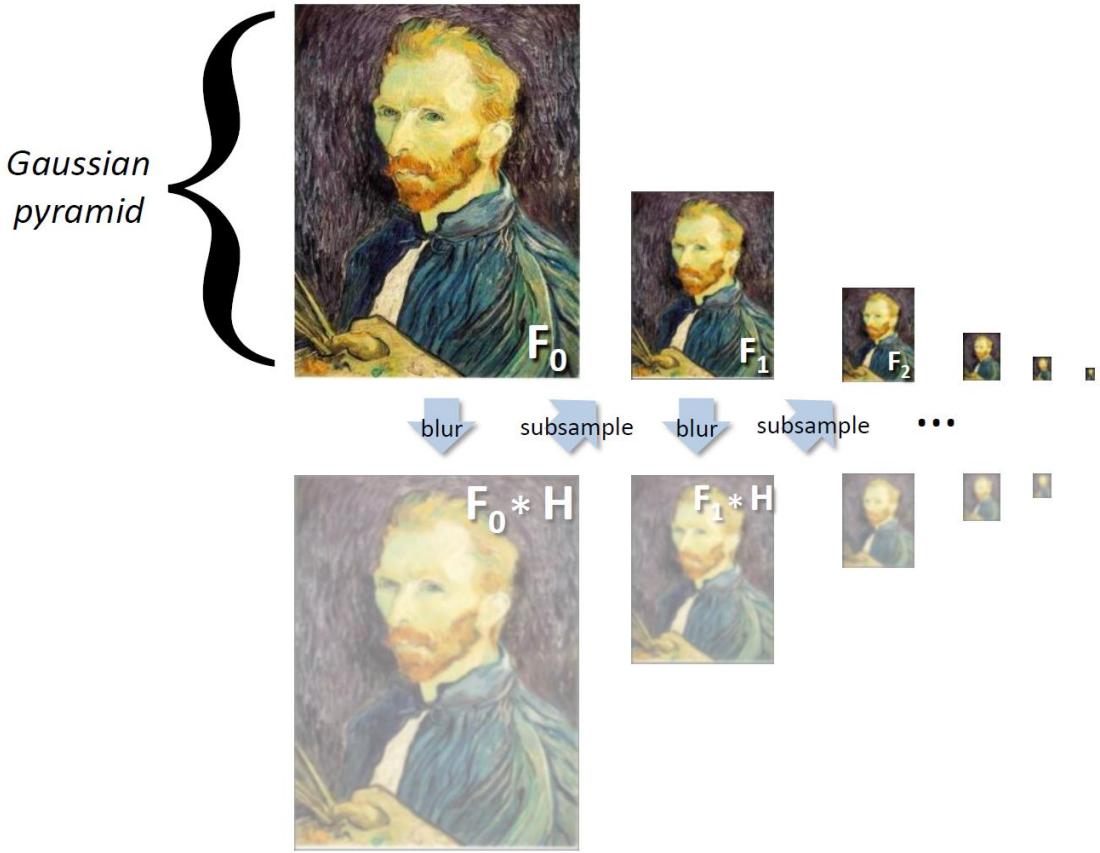
Image extracted from Wikimedia

Exercise 2.A

- Example of a 4-level Gaussian pyramid:



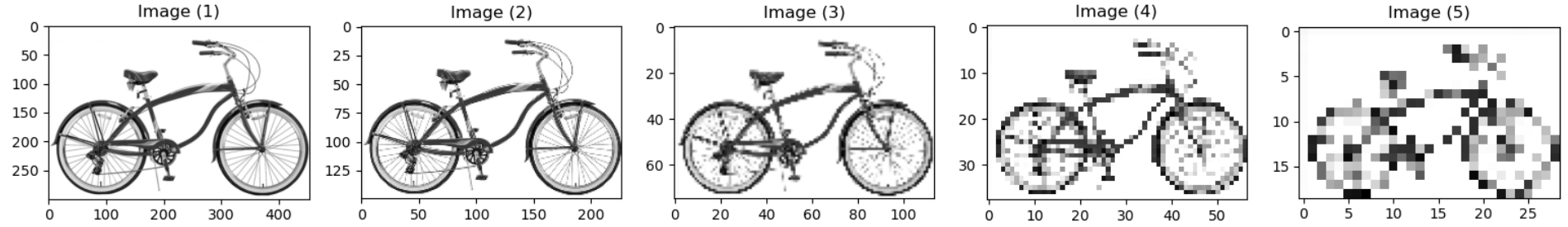
Exercise 2.A


Gaussian pyramid
F_0
F_1
F_2
blur
subsample
blur
subsample
$F_0 * H$
$F_1 * H$

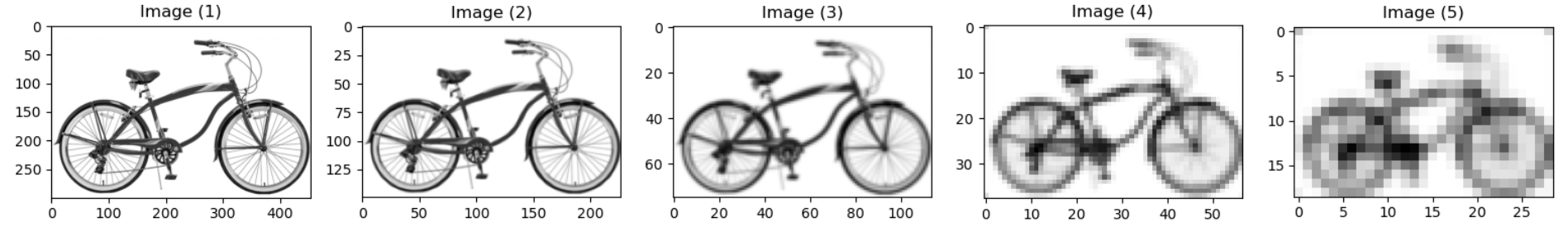
Exercise 2.A

- Effect of smoothing → how does this affect in terms of high frequencies?

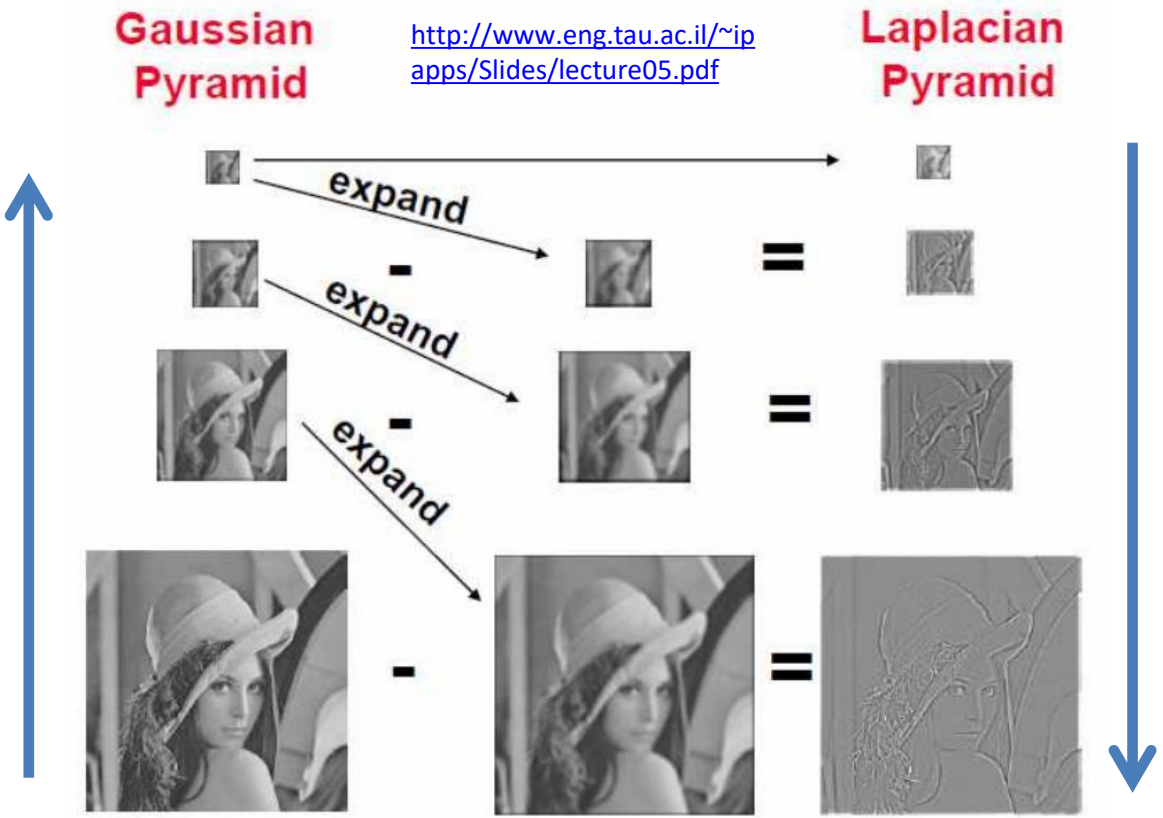
No Gaussian smoothing. Only sub-sampling, keeping the even rows/columns (aliasing):



Including Gaussian smoothing:



Exercise 2.B

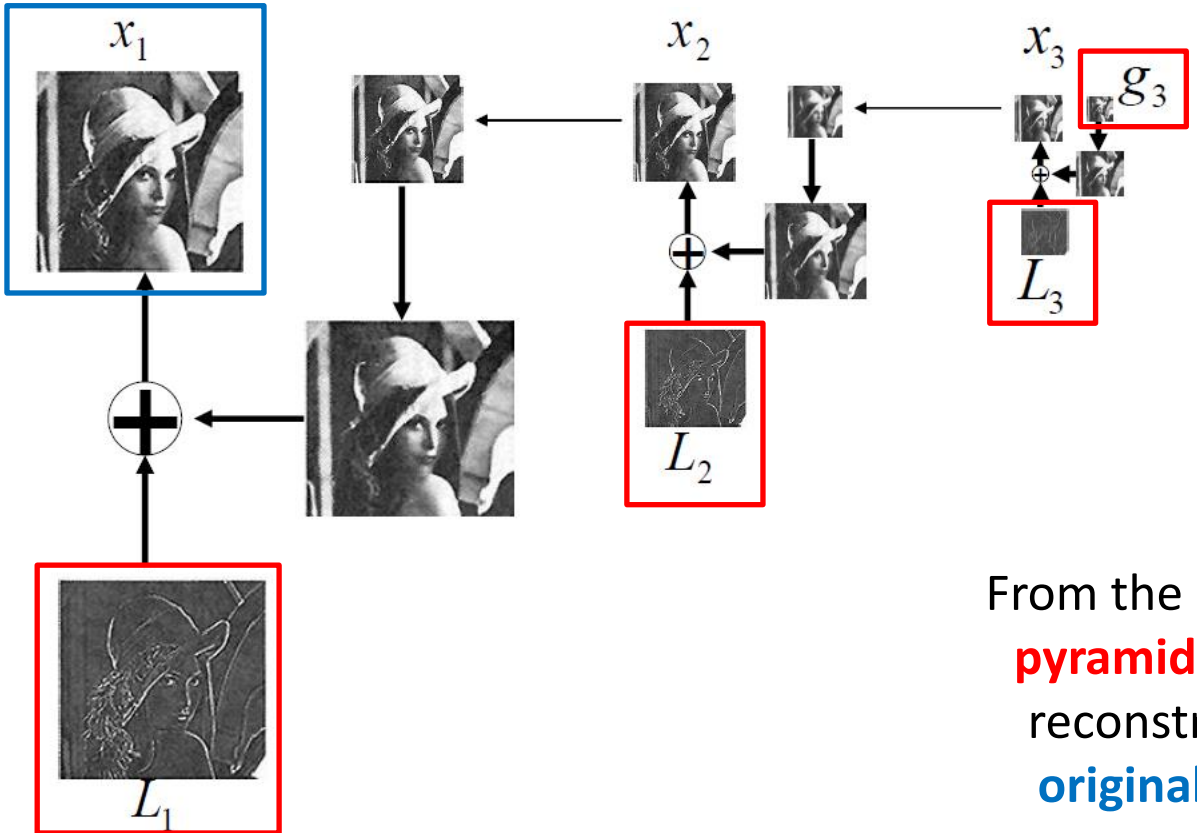


http://www.eng.tau.ac.il/~ipapps/Slides/lecture05.pdf

1) the smallest image produced by the Gaussian Pyramid is taken as the starting point.
2) we expand, calculate the difference, and we have the L_i level.
3) We move to the next level of the Gaussian pyramid, expand and calculate the difference. And we have the L_{i-1} level of the Laplacian Pyramid.
4) And so on...

Exercise 2.C

- We reconstruct x_1 from L_1, L_2, L_3 y g_3



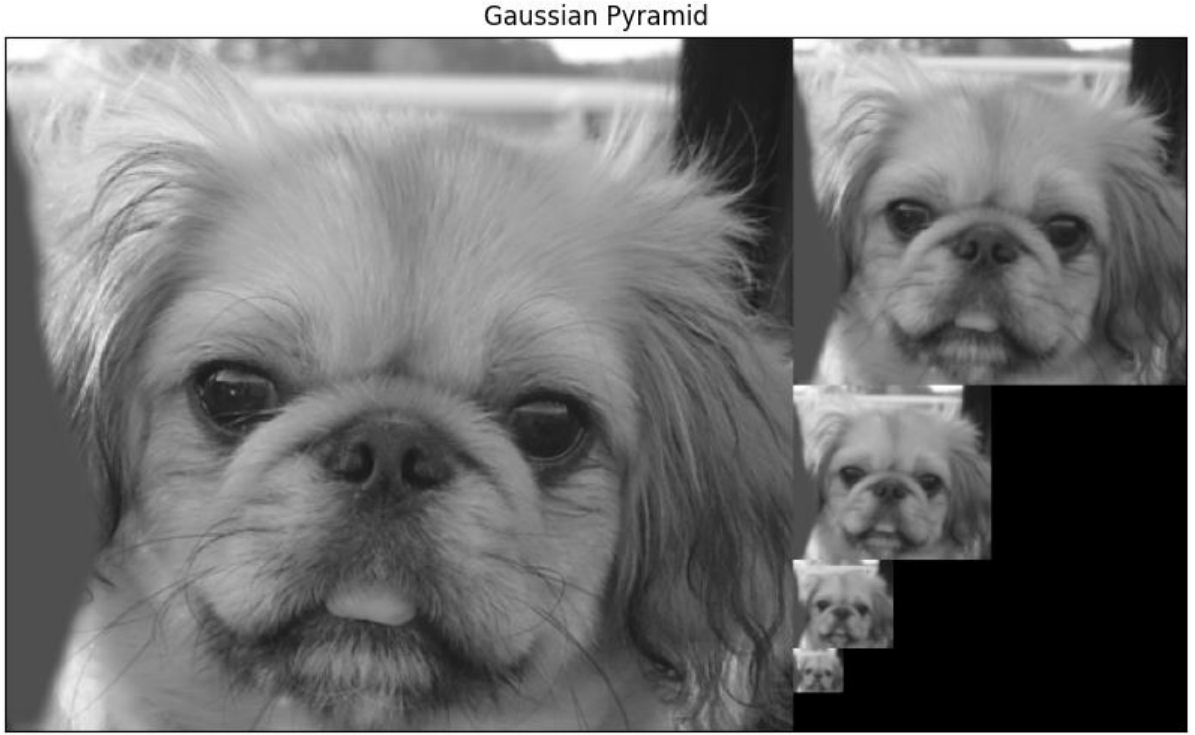
From the **Laplacian pyramid**, we can reconstruct the **original image**

Exercise 2.C

- Is it possible to perfectly reconstruct the original image from a Laplacian pyramid? **Yes!**
- What does this perfect reconstruction depend on: the particular sigma employed, the interpolation used,…? **Mainly on how well you code it!** ☺

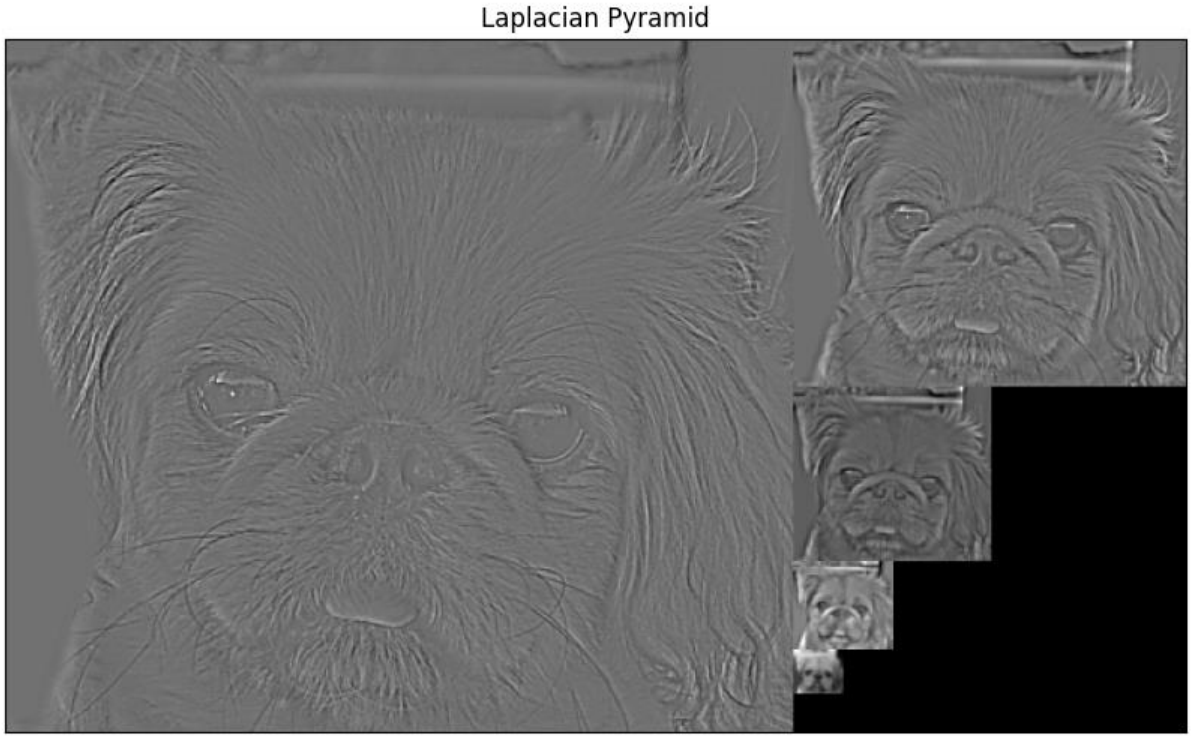
Exercise 2.C

Gaussian pyramid with sizeMask=7 and 4 levels



Exercise 2.C

4-level Laplacian pyramid



Exercise 2.C

Original Image VS Reconstructed image

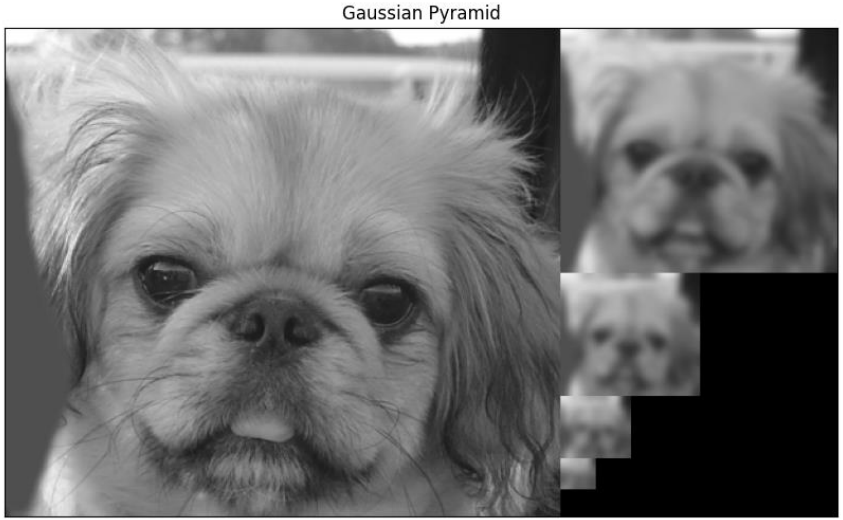


Identical! All pixels are the same!!

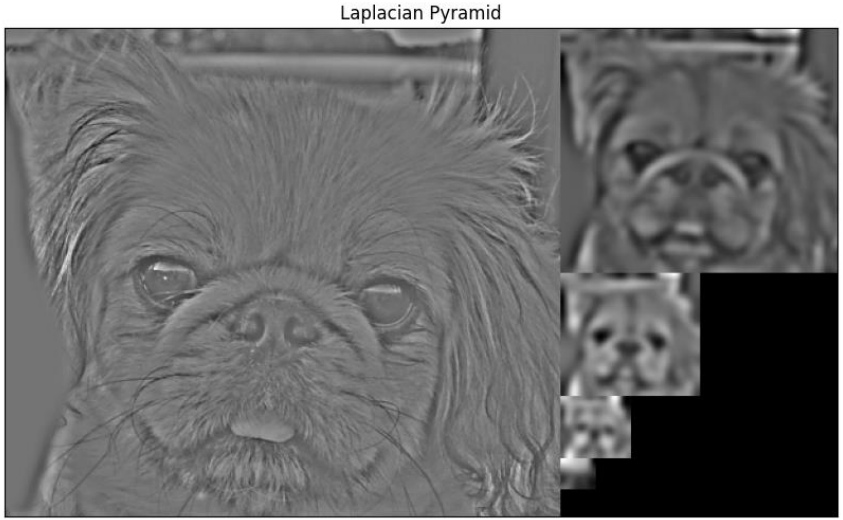
Exercise 2.C

- Note:
  - Sigma does not influence the reconstruction!

Gaussian pyramid (kSize=31)



Laplacian pyramid (ksize=31)

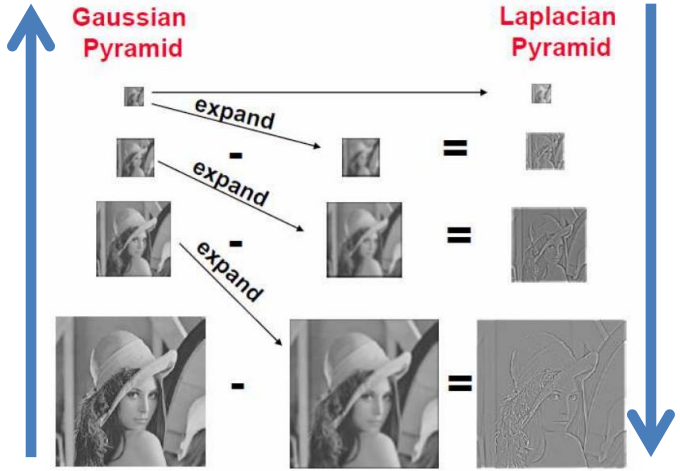


Exercise 2

```
pyramidGauss(im, sizeMask, nlevel):
    # We compute the Gaussian mask

    pyramid = []
    # We keep the original image as pyramid base (level-0)
    pyramid.append(im)
    for i in range(nlevel):
        # We create a new level by smoothing the previous one
        # and keeping half of rows and columns

    return vim #list of images
```




```
pyramidLap(im, sizeMask, nlevel, flagInterp):
    # Compute Gaussian pyramid from input image (im)

    for i in range(nlevel):
        # Resize i+1 level from Gaussian pyramid to the size of i level (cv2.resize) using flagInterp interpolation
        # Compute gaussian_pyramid[i] - expanded_image

    # Keep the last/smallest level of the Gaussian pyramid to be able to reconstruct the original image

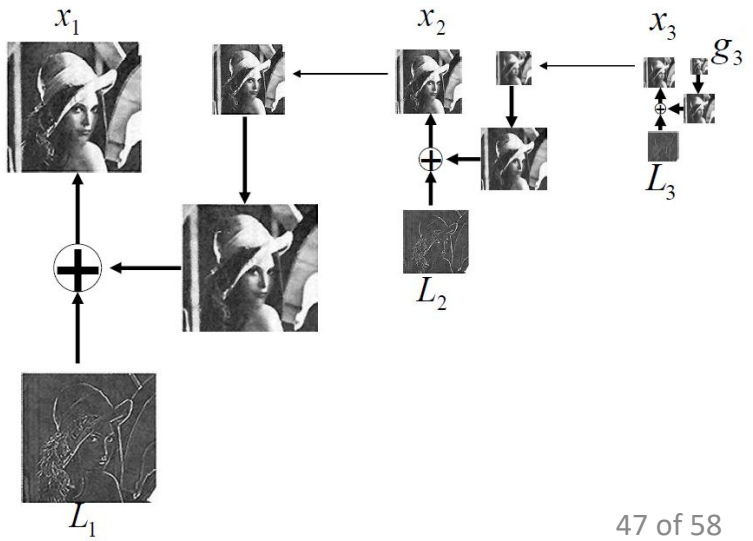
    return vimL #list of images
```

Exercise 2

```
reconstructLap(pyL, flagInterp):
    # Start from the last level of the Laplacian pyramid

    # Go through the Laplacian pyramid, from the top, applying the algorithm:
          # Expand the level to the size of the immediately lower level (cv2.resize) using the
          interpolation determined in flagInterp (bilinear)
          # Compute laplacian_pyramid[i] + expanded_image

    return im #return reconstructed image
```



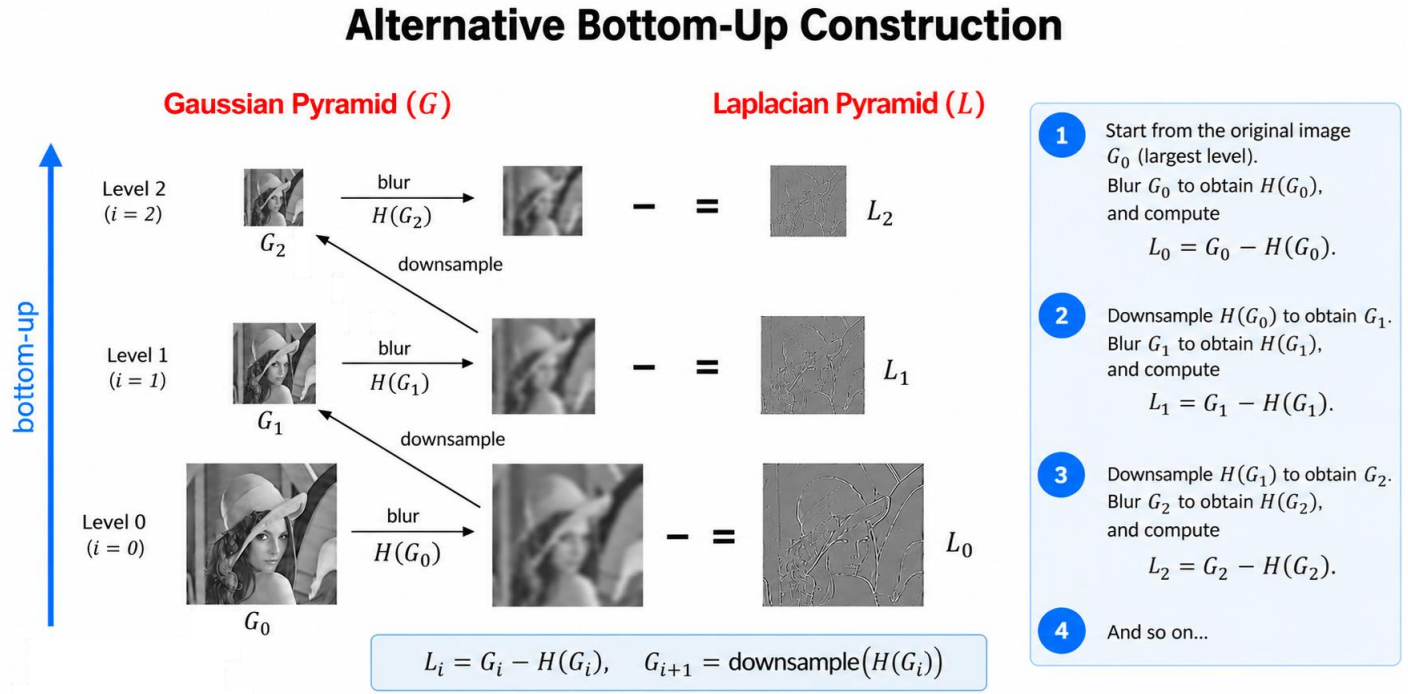
Exercise 2.D

- What if we use different expansion strategies when creating the Laplacian pyramid and when reconstructing the original image?

- If we don’t smooth the Gaussian pyramid, can we still perfectly recover the original image from the Laplacian pyramid?

Exercise 2.D

- Now let us try to build the Laplacian pyramid “bottom-up”, together with the Gaussian pyramid. Is this a valid way to construct a Laplacian pyramid? Would it still allow us to reconstruct the original image?



This figure was created with the assistance of ChatGPT (GPT-5.6 Sol).

Exercise 3

- A. Oliva, A. Torralba, P.G. Schyns (2006). Hybrid Images. ACM Transactions on Graphics.
  - http://olivalab.mit.edu/hybridimage.htm

- By appropriately **mixing part of the high frequencies of one image with part of the low frequencies of another image, we obtain a hybrid image** that allows for **different interpretations at different distances**.

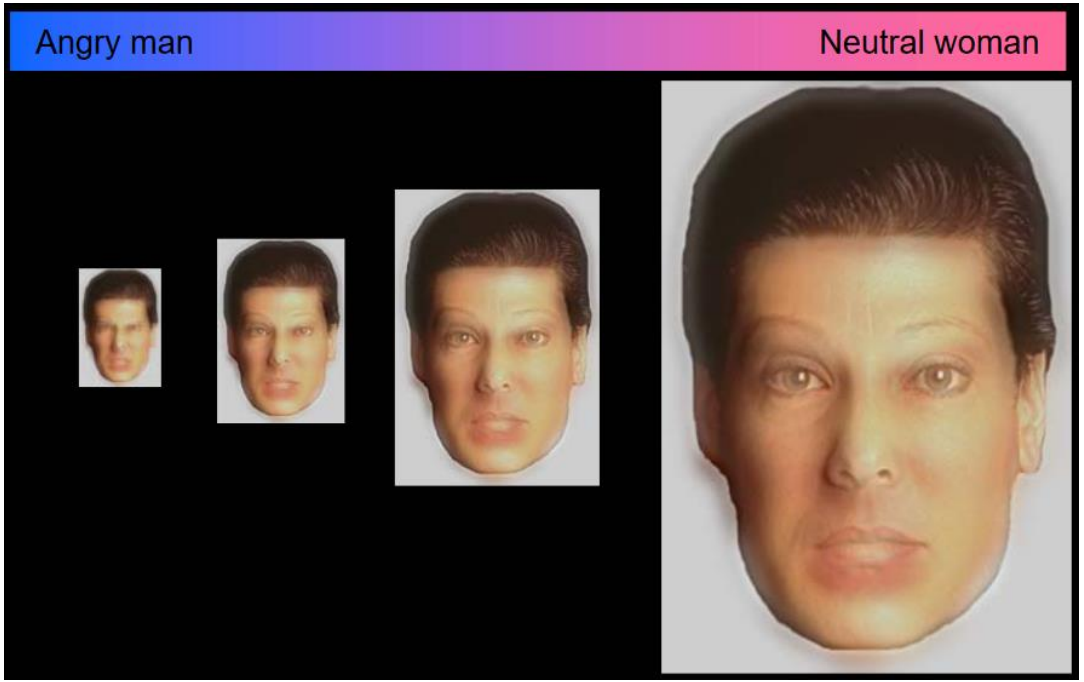


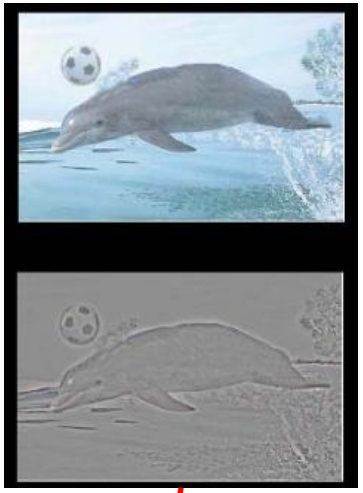
Image taken from
http://olivalab.mit.edu/publications/Talk_Hybrid_Siggraph06.pdf

Exercise 3

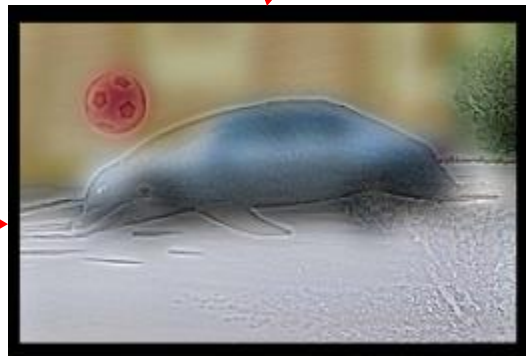
- **You have to implement a function that generates the low and high frequency images from the image pairs.**

Low frequencies predominate at long distances, while high frequencies predominate at short distances

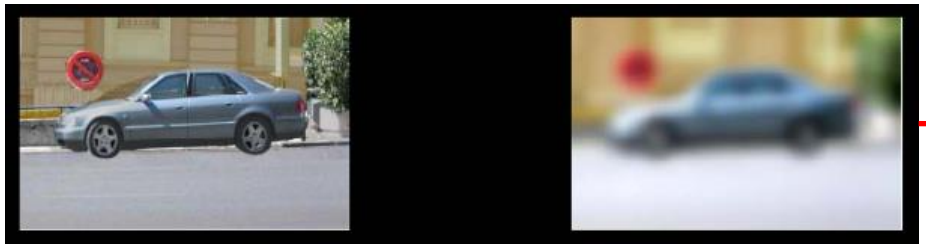
High frequency input image



Hybrid image



Low frequency input image



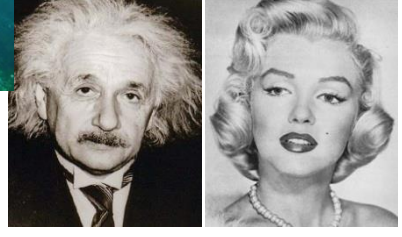
Exercise 3

- **If the effect is achieved, it can be seen in the Gaussian pyramid**
  - You do not need to move away from the computer to see it!
- If the effect is not achieved, the exercise is not correct!
  - E.g. if one figure always clearly predominates over the other
- Choice of high and low frequencies images:
  - High: the one with the most accentuated edges
  - Low: the one with a softer appearance

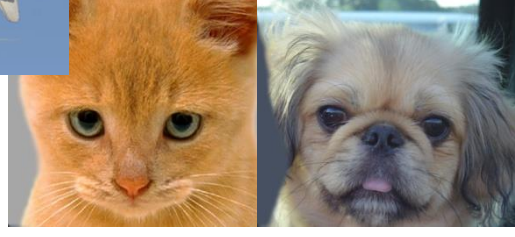












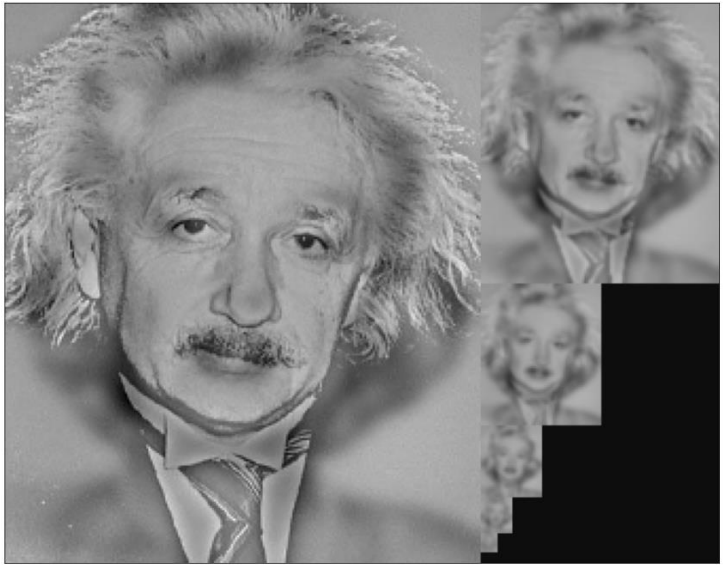
Exercise 3

- How to proceed:

1) Low frequencies: smooth it heavily (until only almost a spot without details remains).

2) High frequencies: calculate difference between the original and its smoothed version.

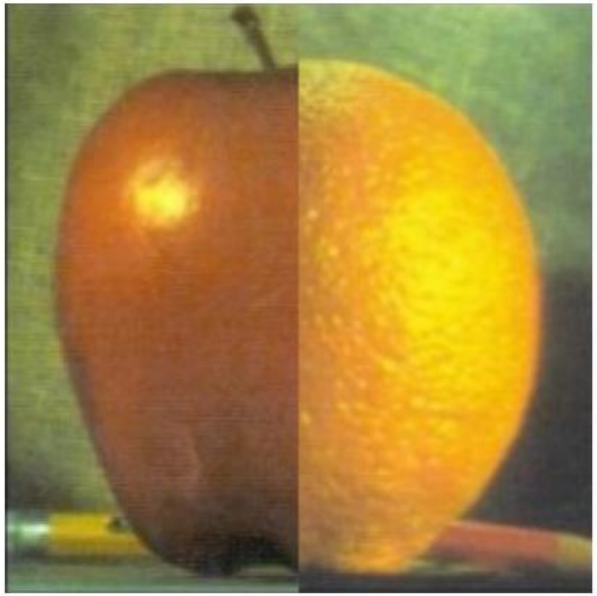




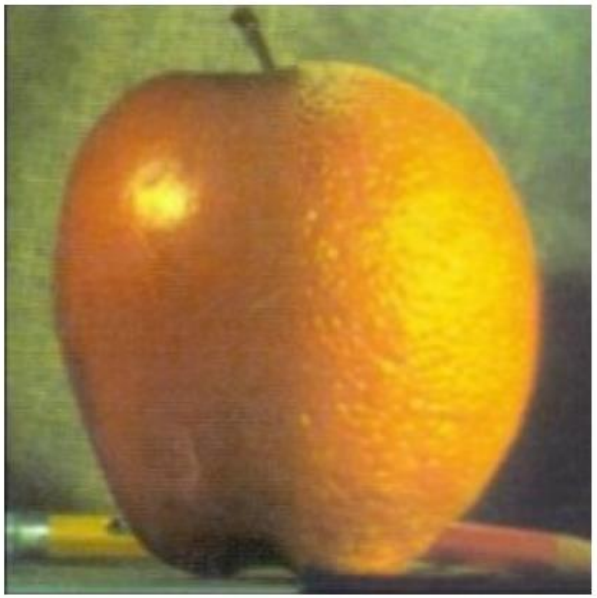
IMPORTANT: It is necessary to establish an "objective" strategy for selecting the high and the low frequencies images. Maybe the Fourier Transform can help you here?

Exercise 4

- Pyramid Blending







Burt and Adelson, “A multiresolution spline with application to image mosaics”, ACM Transactions on Graphics, 1983, Vol.2, pp.217-236.

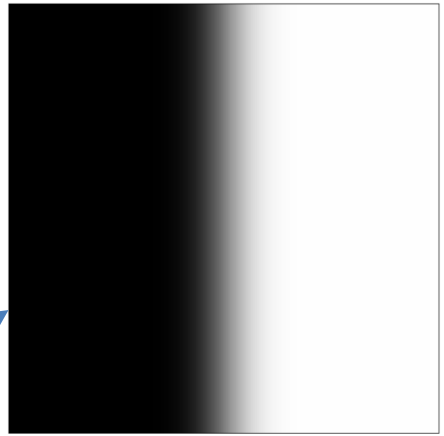
Exercise 4

- Pyramid Blending

1) Read two color images (*A* and *B*)

2) Generate the Laplacian pyramids for *A* (L_A) and *B* (L_B). To do so, you need to compute their Gaussian Pyramids as well.

3) Build a Gaussian pyramid (G_R) from the desired region/mask (R). To get a possibly better final result, this initial input mask (R) can also be smoothed.



4) Mix both laplacians using G_R as a weighted mask

$$L(i,j) = G_R(i,j)L_A(i,j) + \big(1 - G_R(i,j)\big)L_B(i,j)$$

5) Start from this new blended/merged pyramid (L), and reconstruct/collapse the "original" image



Note: you cannot use *pyrUp* and *pyrDown* in this exercise.

Useful reference: https://becominghuman.ai/image-blending-using-laplacian-pyramids-2f8e9982077f

Final Remarks

- Remember to
  - Check the documentation:
    - https://docs.opencv.org/5.0
  - **Work on the discussion of results and explanation of the decisions taken, so that everything you do is clear and well justified**.
    - It's about making it easier for the reader to understand the work (and highlighting your efforts and knowledge)
      → **I cannot evaluate things that are not in the report!**

Final Remarks

- Figures must be easy to understand:
  - Appropriate resolution and size
  - Suitable title
  - They should be commented in a text cell (and, if they are not explicitly demanded in the exercise statement, their existence should be justified in some way)

Final Remarks

- About the discussion/explanation/analysis:
  - Check Ejemplo_Otsu.ipynb
  - It goes in text cells, not in code cells (as code comments do)
  - One should not
    - deviate from the point
    - be excessively brief
  - Do not hesitate to
    - link it to theory in the best way you can
    - incorporate additional experiments that you consider appropriate to demonstrate your hypotheses or to delve into each exercise
      - This is not necessary but, obviously, all the students' hard work and effort will be valued positively!
  - Avoid systematic spelling mistakes → they spoil the result!

Computer Vision

Assignment 2: Image Processing

Pablo Mesejo and Siham Tabik
Universidad de Granada
Departamento de Ciencias de la Computación e Inteligencia Artificial



UNIVERSIDAD DE GRANADA

